**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Бэк-энд разработка

Отчет

# Лабораторная работа №6

Выполнил:

Зотеев Максим

БР1.1

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Настроить сервис мониторинга микросервисов средствами Grafana (связка Grafana + Loki + Promtail + Prometheus). Реализовать дашборд, покрывающий: 1) общее потребление CPU, диска, RAM и сети на ВМ; 2) healthcheck для всех микросервисов, показывающий, все ли сервисы доступны; 3) просмотр логов со всех контейнеров, запущенных на ВМ.

Дополнительное задание: настроить alert на изменение healthcheck через отправку почтовых уведомлений; настроить и интегрировать Sentry (self-hosted или облачный).

Ход работы

1. Исходные условия и выбор конфигурации

Мониторится система сервиса аренды недвижимости из ЛР2–ЛР3, развёрнутая на VPS из ЛР4–ЛР5 (Ubuntu 24.04, IP 82.115.222.101): пять сервисов на Node.js, PostgreSQL и RabbitMQ в отдельном compose-проекте. Ресурсы сервера малы: 1 vCPU, 709 МиБ RAM, 1 ГиБ swap, диск 8,8 ГБ, на котором перед началом работы было свободно 1,5 ГБ (2,1 ГБ после очистки кэша сборки Docker). Этим определены все решения ниже: у контейнеров заданы лимиты памяти, метрики собираются раз в 30 секунд и хранятся 2 дня (не более 100 МБ), логи хранятся сутки.

Мониторинг вынесен в собственный compose-проект monitoring в каталоге /opt/monitoring: при деплое приложения используется remove_orphans, и контейнеры в общем проекте были бы удалены. Как и в прошлых работах, всё разворачивается через Ansible, инвентарь, переменные и Vault взяты из labs/lab4/. Стек нужен на время демонстрации: дашборд проверен на сервере и снят на скриншоты ниже, после чего стек удаляется отдельным плейбуком, иначе автодеплою из ДЗ6 не хватит места на диске. Повторно он поднимается одной командой.

2. Структура проекта

```
lab6/
├── ansible.cfg              # инвентарь и пароль Vault из ../lab4
├── monitoring.yml           # развёртывание стека
├── monitoring-remove.yml    # удаление стека, образов и данных
└── roles/monitoring/
    ├── defaults/main.yml    # каталог, подсеть, порт Grafana
    ├── handlers/main.yml    # перезапуск при изменении конфигурации
    ├── tasks/main.yml
    ├── templates/
    │   ├── docker-compose.yml.j2
    │   ├── env.j2                  # пароли из Vault
    │   └── contact-points.yml.j2   # получатель алертов
    └── files/
        ├── prometheus.yml
        ├── blackbox.yml
        ├── loki.yml
        ├── promtail.yml
        └── grafana/         # provisioning и дашборд rental.json
```

Пароль администратора Grafana, логин и пароль SMTP и DSN Sentry лежат в том же зашифрованном group_vars/all/vault.yml. Используемые образы:

```
grafana/grafana-oss:10.4.19-security-01   256 МиБ
prom/prometheus:v3.15.1                   160 МиБ
grafana/loki:3.7.9                        200 МиБ
grafana/promtail:3.6.11                    64 МиБ
prom/node-exporter:v1.12.1                 32 МиБ
prom/blackbox-exporter:v0.29.1             32 МиБ
```

Справа указан лимит памяти контейнера. Promtail снят с развития в пользу Grafana Alloy, но по заданию используется именно он, последняя доступная версия.

3. Метрики ВМ: Prometheus и node-exporter

node-exporter запущен в сети хоста: в сети Docker он отдавал бы счётчики трафика контейнера, а не виртуальной машины. Корень файловой системы смонтирован только для чтения. Чтобы экспортер не был доступен снаружи, он слушает только адрес шлюза сети мониторинга, а роль добавляет правило UFW, разрешающее обращение из этой подсети:

```
node-exporter:
  image: prom/node-exporter:v1.12.1
  network_mode: host
  pid: host
  command:
    - --path.rootfs=/host
    - --web.listen-address=172.31.0.1:9100
  volumes:
    - /:/host:ro,rslave
```

4. Healthcheck сервисов: blackbox-exporter

У всех пяти сервисов есть эндпоинт /health. Их порты не опубликованы, поэтому blackbox-exporter подключён к сети приложения rental_default и обращается к сервисам по именам. Проверка считается пройденной только при статусе 200 и наличии "status":"ok" в ответе. Дополнительно проверяются TCP-порты PostgreSQL и RabbitMQ.

```
modules:
  http_health:
    prober: http
    timeout: 5s
    http:
      valid_status_codes: [200]
      fail_if_body_not_matches_regexp:
        - '"status":"ok"'
```

В Prometheus цель передаётся экспортеру параметром target, а имя сервиса выделяется из адреса в метку service, которую используют дашборд и алерт:

```
- job_name: health
  metrics_path: /probe
  params:
    module: [http_health]
  static_configs:
    - targets:
        - http://api-gateway:8080/health
        - http://identity-service:3001/health
        # ... property-service, rental-service, messaging-service
  relabel_configs:
    - source_labels: [__address__]
      target_label: __param_target
    - source_labels: [__param_target]
      regex: (?:https?://)?([^:]+):.*
      target_label: service
    - target_label: __address__
      replacement: blackbox-exporter:9115
```

5. Логи контейнеров: Promtail и Loki

Promtail находит контейнеры через Docker API (сокет смонтирован только для чтения), поэтому новые контейнеры подхватываются автоматически, без правки конфигурации. Имя контейнера, проект и поток вывода становятся метками:

```
scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
        refresh_interval: 15s
    relabel_configs:
      - source_labels: [__meta_docker_container_name]
        regex: /(.*)
        target_label: container
      - source_labels:
[__meta_docker_container_label_com_docker_compose_project]
        target_label: project
      - source_labels: [__meta_docker_container_log_stream]
        target_label: stream
```

Loki работает одним процессом с хранением на диске и удаляет логи старше суток через compactor. В него попадают логи всех 13 контейнеров: 7 контейнеров приложения и 6 контейнеров мониторинга.

6. Grafana и дашборд

Источники данных, дашборд и алерты описаны файлами provisioning, так что после развёртывания Grafana ничего не нужно настраивать вручную. Порт опубликован только на 127.0.0.1: опубликованные Docker порты обходят UFW. Grafana открывается через SSH-туннель, анонимный доступ разрешён только на чтение.

```
$ ansible-playbook monitoring.yml
$ ssh -L 3000:127.0.0.1:3000 deploy@82.115.222.101
```

Дашборд «Мониторинг сервиса аренды» состоит из трёх блоков: потребление CPU, RAM, swap, диска и сети на ВМ; плитка «Все сервисы» с общим статусом, плитки healthcheck по каждому сервису и история доступности; логи с фильтром по контейнеру и поиском по тексту (рисунок 1). Красные участки истории на рисунке — проверки из раздела 7.

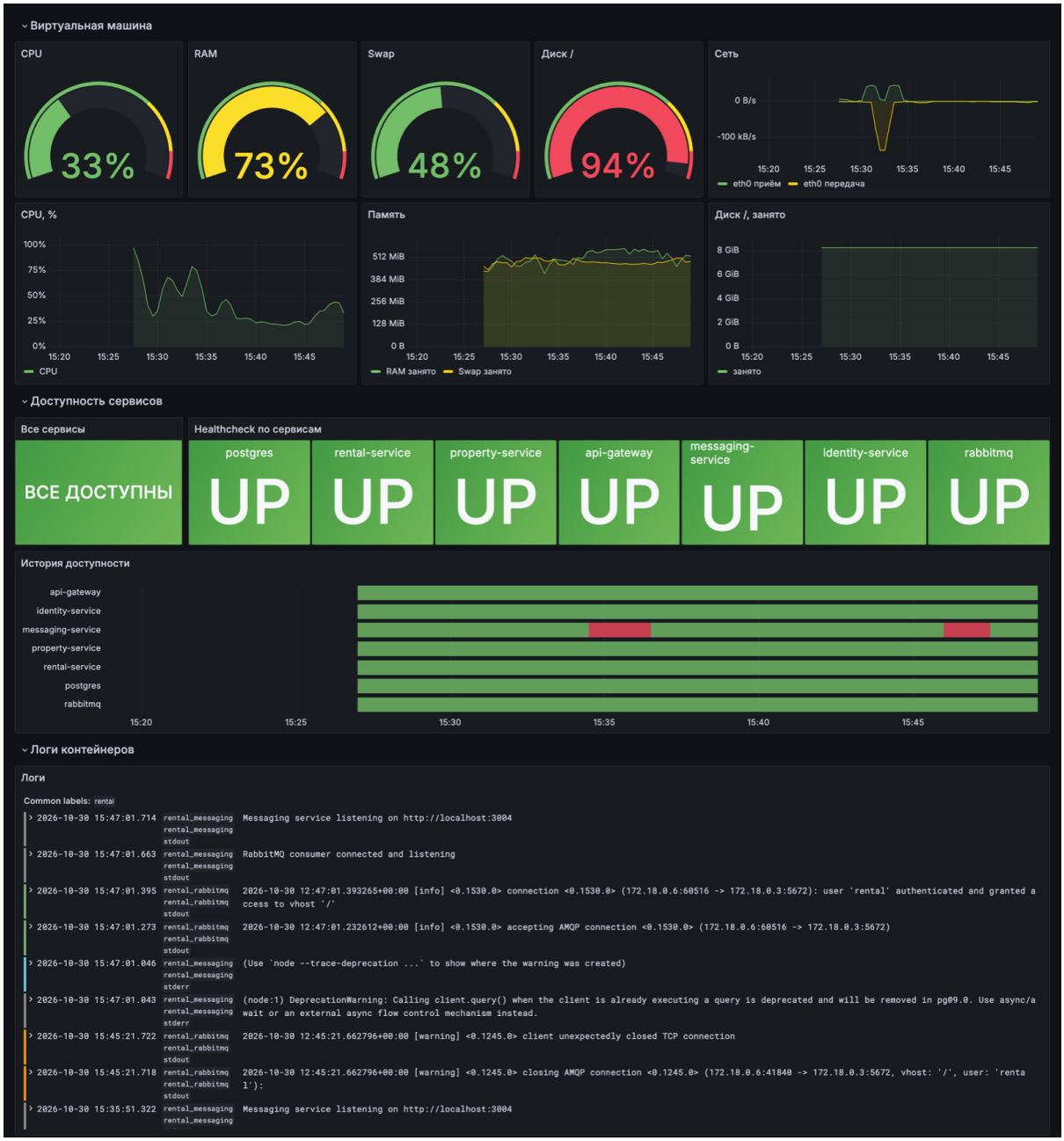



Рисунок 1 — дашборд: ресурсы ВМ, healthcheck и логи контейнеров

7. Почтовый алерт на изменение healthcheck

Правило «Сервис недоступен» проверяет probe_success каждые 30 секунд и срабатывает, если значение меньше 1 в течение 30 секунд. Алерты группируются по сервису, поэтому письмо приходит отдельно на каждый сервис — и о сбое, и о восстановлении. Письма отправляются через SMTP Яндекс Почты с паролем приложения.

```
  - uid: service-down
    title: Сервис недоступен
    condition: C
    for: 30s
    annotations:
      summary: "Не проходит healthcheck: {{ $labels.service }}"
    data:
      - refId: A
        model:
          expr: probe_success{job=~"health|health_tcp"}
      - refId: C
        model:
          type: threshold
          expression: A
          conditions:
            - evaluator: {type: lt, params: [1]}
```

Для проверки на сервере был остановлен контейнер rental_messaging. Примерно через 80 секунд healthcheck стал красным, а алерт перешёл в состояние firing (рисунки 2 и 3). На почту пришло письмо о сбое, а после запуска контейнера — письмо о восстановлении (рисунок 4).




Рисунок 2 — дашборд при остановленном messaging-service

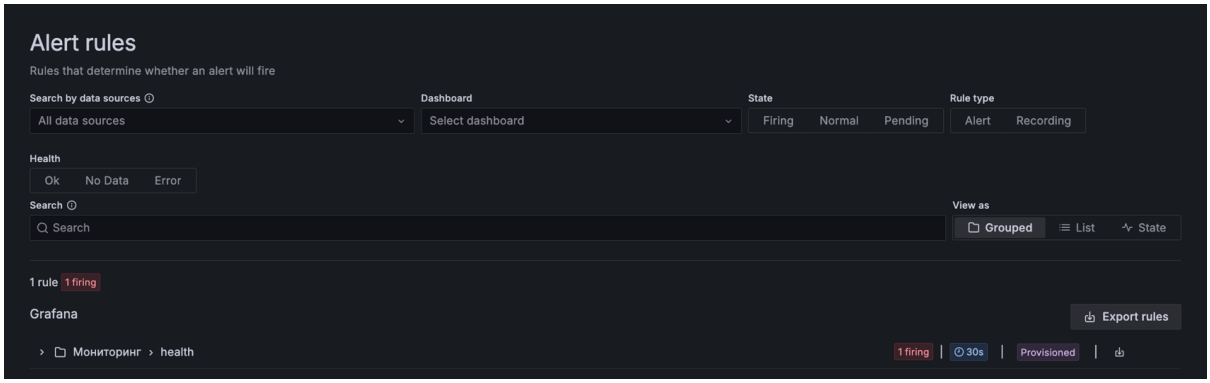



Рисунок 3 — сработавшее правило в Grafana

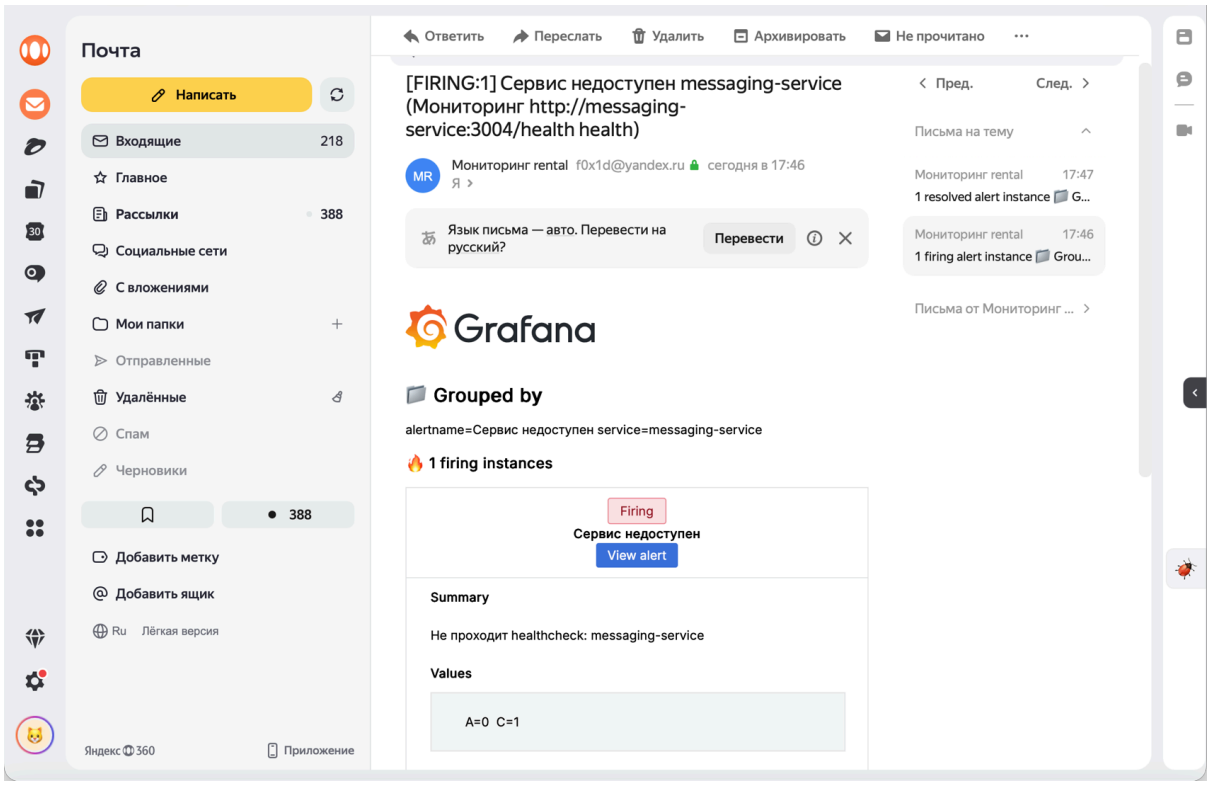



Рисунок 4 — письмо о сбое и письмо о восстановлении в цепочке справа

8. Интеграция с Sentry

Self-hosted Sentry на этом сервере запустить невозможно: ему требуется порядка 16 ГБ оперативной памяти и 20 ГБ диска. Поэтому интегрирован облачный sentry.io, для чего в нём создан проект, а его DSN добавлен в Vault и передаётся сервисам через .env и переменную SENTRY_DSN. Без DSN, например при локальном запуске, отправка отключена.

Использован облегчённый вариант SDK @sentry/node-core/light: только ошибки, без трейсинга и OpenTelemetry. При замере он занял 37 МБ в node_modules против 83 МБ у @sentry/node и на 16 МБ меньше памяти процесса, что заметно на пяти сервисах. Точка входа light доступна через поле exports, поэтому в tsconfig общего пакета и шлюза параметр module изменён на node16.

Инициализация вынесена в общий пакет @rental/shared и вызывается в начале каждого сервиса:

```
export const initSentry = (service: string) => {
  const dsn = process.env.SENTRY_DSN;
  if (dsn) Sentry.init({ dsn, serverName: service });
};

// process.exit оборвал бы отправку, поэтому событие дожидается доставки.
export const reportFatal = async (err: unknown) => {
  Sentry.captureException(err);
  await Sentry.flush(2000);
};
```

В Sentry попадают только непредвиденные ошибки: общий errorHandler отправляет ошибки 500, а ожидаемые HttpError (400, 401, 404) не отправляет. Ошибки запуска сервиса передаются через reportFatal. api-gateway собирается отдельно от @rental/shared, поэтому SDK подключён к нему напрямую, а в Sentry уходят ошибки проксирования:

```
const proxyDefaults: Partial<Options> = {
  // ...
  on: { error: (err) => Sentry.captureException(err) },
};
```

Логика errorHandler проверена локально на заглушке Sentry: ошибка 500 и ошибка запуска были отправлены, а HttpError 404 — нет. На сервере проверка совмещена с разделом 7: запрос к остановленному messaging-service через шлюз вернул 504, а в Sentry появилась ошибка getaddrinfo ENOTFOUND messaging-service с адресом запроса и стеком вызовов (рисунок 5).

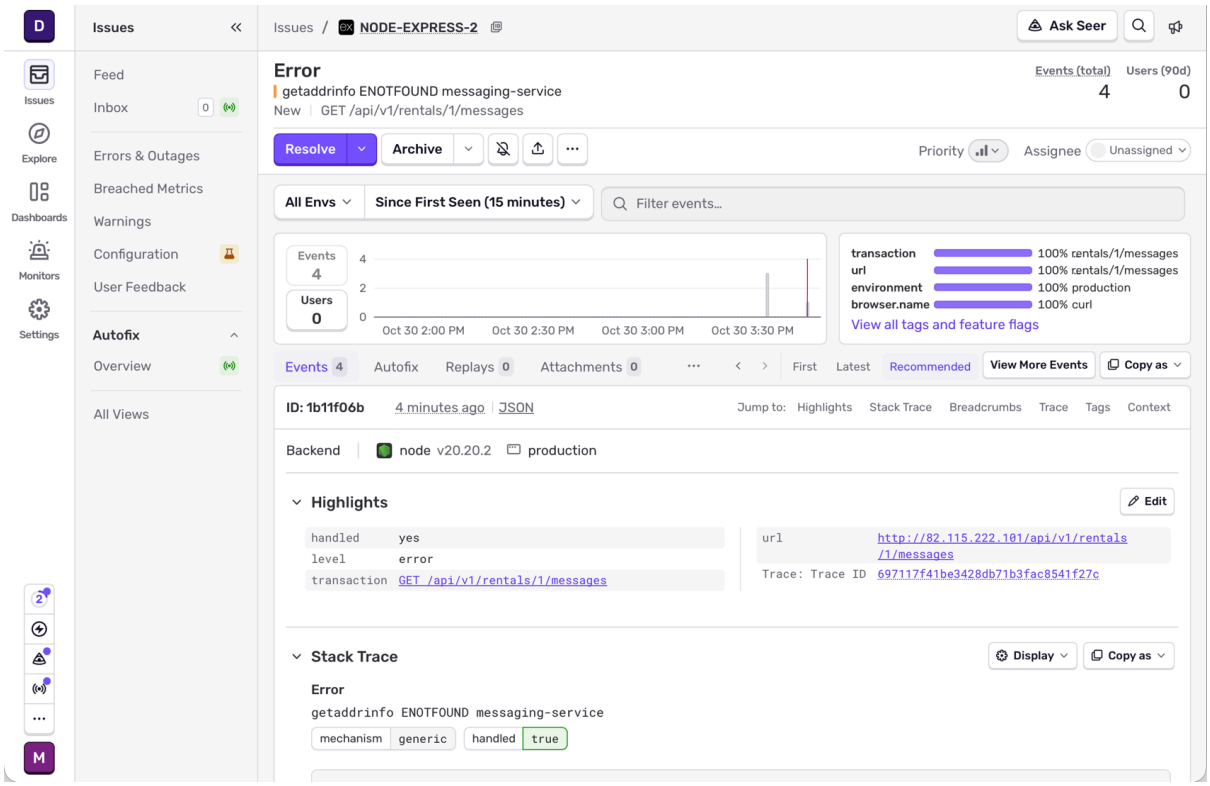



Рисунок 5 — ошибка api-gateway в Sentry

9. Проблемы при развёртывании

Заполнение диска. Первый запуск со свежей Grafana 13 занял весь свободный диск: только её образ весит 1,3 ГБ, контейнеры Prometheus и Loki стали перезапускаться с ошибками, а Ansible упал с No space left on device. Приложение при этом продолжало отвечать, стек был остановлен, а образ удалён. Использована Grafana 10.4, образ которой намного меньше, а также очищены кэш сборки и кэш apt. В итоге на время работы стека оставалось свободно около 500 МБ.

Блокировка SQLite. При первом запуске на одном vCPU Grafana не смогла сохранить права на дашборд (database is locked), и анонимный доступ возвращал 403. Включён режим WAL (GF_DATABASE_WAL), данные Grafana пересозданы, после чего дашборд открывается.

Отказ Яндекса. Первая отправка письма завершилась ошибкой 535 This user does not have access rights to this service. В настройках почты был выключен доступ почтовых программ. После его включения письма стали приходить, проверка из раздела 7 повторена.

10. Проверка

Повторный запуск плейбука ничего не меняет:

```
rental-vps                  : ok=9    changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

Все контейнеры работали, использование памяти после работы с дашбордом (без учёта вытесненного в swap):

```
monitoring_grafana        27.68MiB / 256MiB
monitoring_promtail        5.69MiB / 64MiB
monitoring_node_exporter   6.95MiB / 32MiB
monitoring_loki           44.99MiB / 200MiB
monitoring_prometheus     12.18MiB / 160MiB
monitoring_blackbox        5.24MiB / 32MiB

               total        used        free
Mem:             709         479          51
Swap:           1023         508         515
/dev/vda1       8.8G  7.9G  502M  95% /
```

После проверок стек удалён плейбуком monitoring-remove.yml вместе с образами, томами и правилом UFW. Приложение с подключённым Sentry продолжает работать:

```
$ ansible-playbook monitoring-remove.yml
rental-vps                  : ok=4    changed=3    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0

/dev/vda1       8.8G  6.3G  2.1G  76% /

$ curl http://82.115.222.101/health
{"status":"ok","service":"api-gateway"}
```

Заключение

В рамках ЛР6 для сервиса аренды недвижимости настроен мониторинг на связке Grafana, Prometheus, Loki и Promtail, развёрнутый через Ansible отдельно от приложения. Дашборд показывает потребление CPU, RAM, диска и сети на ВМ, доступность всех сервисов через healthcheck и логи всех контейнеров. Дополнительно

настроены почтовые алерты при изменении healthcheck и интеграция с облачным Sentry: оба механизма подтверждены остановкой messaging-service. На слабом сервере основным ограничением оказался диск, поэтому стек работает с лимитами памяти и коротким хранением данных и запускается только на время использования.